

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

Trabajo Final Integrador

E-commerce con reserva por WhatsApp

Caso de aplicación: Librería Mi Sueño

Arquitectura técnica, fundamentos de decisión y análisis de viabilidad

Autores:

- Franco Ghilardi
- Agustin Chiaravalloti

Tutor: Juan Ignacio Schiavonni

Grupo: 208

Fecha: 30 de Agosto 2026

Índice

Registro de entregas

1. Identificación del problema

- 1.1 Contexto
- 1.2 Problemas identificados
- 1.3 Objetivos del proyecto

2. Propuesta de solución

- 2.1 Visión general
- 2.2 Alcance de la primera implementación (MVP)
- 2.3 Mejora futura (fuera del alcance del MVP)
- 2.4 Relación entre el MVP y la mejora futura

3. Definición del stack tecnológico

- 3.1 Arquitectura de referencia

4. Toma de decisiones de stack fundamentadas

- 4.1 Frontend: Next.js en Vercel
- 4.2 Backend: NestJS separado, alojado en Render
- 4.3 ORM: Drizzle en lugar de Prisma
- 4.4 Hosting del backend: Render
- 4.5 Base de datos y almacenamiento: Supabase
- 4.6 Resumen de decisiones

5. Análisis de viabilidad

- 5.1 Viabilidad técnica
- 5.2 Riesgos y mitigaciones
- 5.3 Conclusión

Anexos

- 1.1 Repositorio de GitHub

Registro de entregas

Este Trabajo Final Integrador se desarrolla de forma incremental. Cada entrega agrega o revisa secciones del documento; las anteriores permanecen como base de las siguientes salvo que se indique explícitamente una revisión.

Entrega	Contenido
1	Identificación del problema, propuesta de solución (MVP y mejora futura), definición y justificación del stack tecnológico, análisis de viabilidad inicial.
2	
3	
4	

1. Identificación del problema

1.1 Contexto

Librería Mi Sueño es un comercio de barrio en Mendoza que vende libros, útiles escolares, artística y regalería, y ofrece servicios de impresión, fotocopiado, anillado, plastificado y Rapipago. Toda la operación comercial hoy es presencial o telefónica: no existe presencia del negocio en internet más allá del contacto directo por WhatsApp.

El circuito de venta actual es enteramente manual: el cliente pregunta uno por uno qué productos hay disponibles, a qué precio y si hay stock, y el vendedor responde de memoria o revisando físicamente el local. No hay un catálogo consultable fuera del horario de atención ni una forma de que el cliente arme un pedido antes de escribir.

1.2 Problemas identificados

- Sin catálogo público: el negocio no tiene ninguna superficie donde un cliente pueda ver qué se vende y a qué precio, sin tener que preguntar.
- Fricción en la consulta: cada pedido implica una conversación de ida y vuelta por WhatsApp para relevar disponibilidad y precio, lo que consume tiempo del vendedor y desalienta pedidos grandes o de único ítem.
- Nula visibilidad en buscadores: al no existir un sitio, la librería no aparece en Google ni es descubrible por clientes nuevos del barrio o de Mendoza en general.
- Dependencia total del trato presencial: no hay ningún canal asincrónico donde el cliente arme y confirme un pedido fuera del horario de atención.
- Sin registro único de la oferta: no existe un listado único y actualizado de productos, precios y stock; la información vive en la cabeza del personal y en el local físico.
- Crecimiento limitado por capacidad de atención humana: cada consulta manual escala linealmente con el volumen de clientes, sin apalancamiento tecnológico.
- Sin autonomía de gestión: aun si existiera un catálogo online, el negocio necesita poder mantenerlo (altas, bajas, precios) sin depender de un desarrollador para cada cambio.

1.3 Objetivos del proyecto

A partir de estos problemas, el proyecto define los siguientes objetivos:

1. Publicar un catálogo online navegable, con buscador, filtro por categorías y ficha de producto, accesible desde celular en cualquier momento.
2. Permitir que el cliente arme un pedido (carrito) sin intervención del vendedor y lo derive a WhatsApp ya armado, para concretar la reserva.
3. Dar a la librería una gestión de contenido propia, mediante un panel de administración con login, para que el negocio cargue y mantenga el catálogo sin depender de un desarrollador para cada cambio.
4. Diseñar la arquitectura para que, en una etapa posterior, se puedan incorporar pagos online y gestión de pedidos sin reconstruir lo ya hecho.
5. Sostener una base técnica sólida y escalable, con separación clara entre las capas de presentación, lógica de negocio y datos.

2. Propuesta de solución

2.1 Visión general

La solución es un e-commerce de catálogo con reserva por WhatsApp: el cliente navega productos, arma un carrito y el sistema genera un mensaje de WhatsApp con el detalle del pedido para que el vendedor lo confirme. El negocio administra el catálogo desde un panel propio, con login, sin depender de terceros para cada cambio.

El proyecto se plantea en dos horizontes: una primera implementación (MVP) que resuelve el problema completo de catálogo, reserva y autogestión de contenido, y una mejora futura que suma pagos online y gestión de pedidos sobre la misma base ya construida.

2.2 Alcance de la primera implementación (MVP)

- **Tienda pública:** home con productos destacados, ofertas y novedades; buscador general y acceso directo al catálogo.
- **Catálogo de productos:** buscador, filtro por categoría, paginado, y cada producto mostrado con foto, nombre y precio.
- **Ficha de producto:** galería de hasta 5 imágenes, título, descripción, disponibilidad, y botones de agregar al carrito y consultar por WhatsApp.
- **Carrito de compras:** accesible desde cualquier página, con cantidades, precios y total; un botón arma el mensaje de reserva y lo envía por WhatsApp.
- **Página institucional:** "Sobre nosotros", con historia e información del local.
- **Encabezado y pie de página:** presentes en todas las pantallas, con navegación, botón de WhatsApp, redes sociales y dirección.
- **Panel de administración:** acceso privado con usuario y contraseña; listado de productos con buscador; alta y edición de productos (nombre, descripción, precio, disponibilidad, hasta 5 imágenes, categoría); marcado de producto como oferta, novedad o destacado; alta, edición y baja de categorías; baja simple (reversible) y baja definitiva de productos y categorías.
- **Base de datos propia:** modelo relacional para productos, categorías, imágenes y usuarios del panel, con un único usuario administrador previsto inicialmente y el modelo preparado para sumar un rol "vendedor" con permisos limitados.
- **Diseño 100 % adaptable:** tienda y panel usables en celular, tablet y computadora.

2.3 Mejora futura (fuera del alcance del MVP)

Quedan fuera de la primera implementación, como mejoras a incorporar más adelante sobre la misma base:

- **Pagos online con Mercado Pago:** tarjeta de crédito o débito con cuotas, dinero en cuenta o transferencia, gestionados íntegramente en el entorno de Mercado Pago.
- **Carrito con doble camino:** "Pagar ahora" (online) o "Reservar por WhatsApp", conviviendo en el mismo carrito.
- **Gestión de pedidos:** cada compra registrada con productos, cantidades, datos del comprador, medio de pago y estado (pendiente, pagado, entregado, cancelado).
- **Stock automático:** descuento de stock al confirmarse un pago, con bloqueo de compra cuando no hay disponibilidad.
- **Avisos por correo:** confirmación de compra al cliente y aviso de venta al negocio.
- **Rol "vendedor" activo:** un segundo usuario con permisos limitados, ya contemplado en el modelo de datos del MVP.

- **Estadísticas básicas:** ventas del período, productos más vendidos y visitas a la tienda.

2.4 Relación entre el MVP y la mejora futura

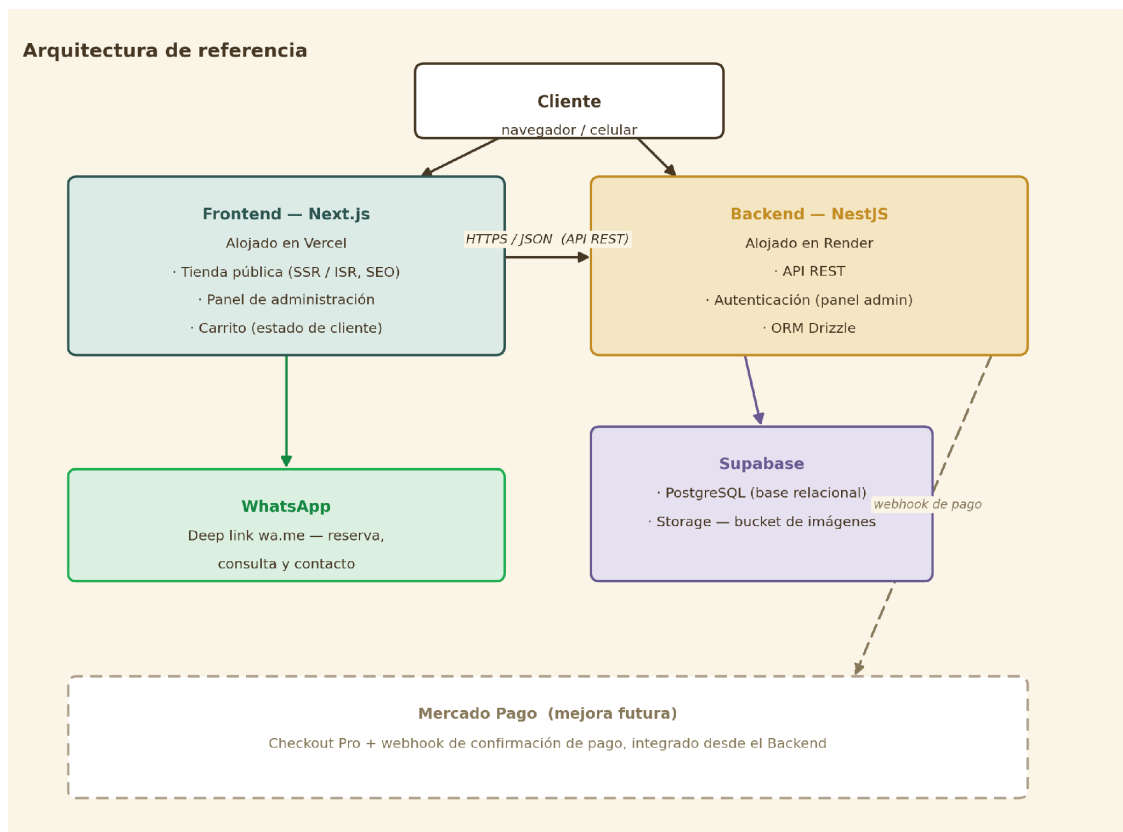
El MVP no es una versión reducida que luego se descarta: es la base funcional y de datos sobre la que se apoya la mejora futura. El modelo de datos del MVP ya incluye las entidades necesarias para pagos y pedidos (ver sección 4.3), y la separación entre frontend y backend definida en la sección 3 permite agregar los endpoints de pago y las pantallas de gestión de pedidos sin modificar la arquitectura ni reescribir lo ya construido.

3. Definición del stack tecnológico

El sistema se construye con una arquitectura desacoplada: un frontend y un backend independientes, cada uno con su propio repositorio, ciclo de despliegue y responsabilidad. Esta separación es un requisito de diseño desde el inicio del proyecto, no una evolución posterior.

Capa	Tecnología	Rol
Frontend	Next.js (React), alojado en Vercel	Tienda pública y panel de administración. SSR/ISR para SEO y velocidad de carga.
Backend / API	NestJS + Drizzle ORM, alojado en Render	API REST propia: catálogo, autenticación del panel, y — en la mejora futura — pedidos y pagos.
Base de datos	PostgreSQL relacional en Supabase	Productos, categorías, imágenes, usuarios y, en la mejora futura, pedidos.
Almacenamiento de archivos	Bucket de Supabase Storage	Imágenes de productos, servidas por URL pública o firmada.
Mensajería con el cliente	WhatsApp vía deep links (wa.me)	Reserva, consulta y contacto, sin costo ni integración paga.
Pagos (mejora futura)	SDK de Mercado Pago, integrado desde el backend Nest	Checkout Pro, confirmación de pago vía webhook contra la API propia.

3.1 Arquitectura de referencia



El frontend (Next.js en Vercel) es responsable exclusivamente de la interfaz: tienda pública y panel de administración. El backend (NestJS en Render) concentra toda la lógica de negocio, la autenticación y el acceso a datos, y expone una API REST versionada que el frontend consume por HTTPS. La base de datos y el almacenamiento de archivos viven en Supabase, fuera de ambos servicios de cómputo. Mercado Pago se integra en la mejora futura, exclusivamente desde el backend.

4. Toma de decisiones de stack fundamentadas

Cada pieza del stack definido en la sección 3 se eligió comparándola contra alternativas concretas. Se documentan acá los motivos, no solo la elección, para que la decisión sea auditable y revisable en entregas futuras.

4.1 Frontend: Next.js en Vercel

- SEO y velocidad de carga: crítico para que las fichas de producto aparezcan en Google y para que la tienda cargue rápido en conexiones de celular, escenario de uso principal del negocio.
- Un solo framework para tienda pública y panel de administración, lo que reduce la superficie de mantenimiento del lado del cliente.
- Ecosistema grande de React, con componentes (shadcn/ui, Radix) disponibles para acelerar la construcción del panel de administración.
- Vercel da CDN global, HTTPS, previews por rama y despliegue continuo desde GitHub sin configuración adicional, con plan gratuito suficiente durante el desarrollo.

Alternativas descartadas: una SPA sin metaframework (Vite) simplifica el build pero pierde el SSR/ISR que sostiene el SEO requerido. Remix es comparable en capacidades, pero tiene un ecosistema de componentes y de hosting gestionado más chico que el de Next.js/Vercel para este tamaño de equipo.

4.2 Backend: NestJS separado, alojado en Render

El backend se construye como una aplicación independiente del frontend, no como funciones o rutas dentro del mismo proyecto Next.js. Los motivos:

- **Separación de responsabilidades y de escalado:** el frontend (contenido, SEO, renderizado) y el backend (lógica de negocio, validaciones, integraciones de pago) tienen perfiles de carga y de despliegue distintos. Separarlos permite escalar o reiniciar uno sin afectar al otro, y evita que un pico de tráfico de la tienda pública compita por recursos con la API.
- **Estructura para crecer en alcance:** a medida que se suman autenticación, roles, webhooks de Mercado Pago y lógica de stock en la mejora futura, un backend estructurado con módulos, inyección de dependencias y testing de primera clase (NestJS) escala mejor en mantenibilidad que funciones sueltas sin framework.
- **Portabilidad:** un backend Nest corriendo como servicio Node estándar (no como función serverless atada a un proveedor) se puede mover a cualquier plataforma con soporte de contenedores o de Node sin reescribir código.
- **NestJS puntualmente:** impone una estructura modular (controllers, services, DTOs, guards) con soporte nativo de TypeScript y decoradores de validación (class-validator), que mapea limpio contra el dominio del proyecto (productos, categorías, usuarios, y en la mejora futura pedidos y pagos).

Alternativas consideradas y descartadas: Express o Fastify sin framework encima son más livianos pero delegan toda la estructura (módulos, inyección de dependencias, validación) al criterio de cada desarrollador, lo que en un proyecto de varias entregas tiende a acumular inconsistencia. Construir el backend como rutas de API dentro del propio proyecto Next.js se descartó por el punto de escalado y mantenibilidad recién explicado: esa alternativa mezclaría dos responsabilidades distintas en un solo despliegue.

4.3 ORM: Drizzle en lugar de Prisma

Criterio	Drizzle	Prisma
Modelo mental	SQL tipado: las queries se escriben muy cerca del SQL real, con autocompletado y tipos inferidos.	DSL propio (schema.prisma) que genera un cliente; abstrae más el SQL subyacente.
Rendimiento / arranque	Sin motor de query engine binario aparte; queries compiladas a SQL directo, arranque más liviano.	Depende de un query engine empaquetado junto a la app; arranque algo más pesado.
Control fino	Alto: útil para las consultas de catálogo con filtros, categorías y paginado.	Bueno para CRUD estándar; consultas muy específicas a veces requieren SQL crudo igual.
Migraciones	drizzle-kit genera SQL versionado, legible y editable a mano si hace falta.	Prisma Migrate, maduro y muy usado, pero el SQL generado es menos editable en la práctica.
Curva de adopción	Requiere pensar en términos de SQL relacional.	Más rápido de empezar para quien no viene de SQL.

Se elige Drizzle porque el modelo de datos del proyecto (productos, categorías, imágenes, usuarios y, en la mejora futura, pedidos e ítems de pedido) es relacional sin operaciones exóticas, y porque se prioriza un runtime liviano en Render y queries explícitas por sobre la velocidad de adopción que da la abstracción de Prisma. La contrapartida —una curva de entrada algo mayor— se considera aceptable dado que el backend se construye con TypeScript estricto en todo el stack.

4.4 Hosting del backend: Render

- Soporta servicios Node de larga duración (no solo funciones serverless de ejecución corta), lo que encaja con NestJS: un proceso persistente con su propio ciclo de vida, conexiones a base de datos con pool y, en la mejora futura, procesamiento de webhooks que necesita responder rápido sin cold start recurrente.
- Despliegue continuo desde GitHub, HTTPS gratuito y variables de entorno gestionadas — mismo flujo operativo que Vercel para el frontend, sin curva de aprendizaje adicional.
- Modelo de precios simple y predecible por servicio, más fácil de presupuestar que un modelo de facturación por uso.

Alternativas consideradas: Railway ofrece una experiencia similar con precios por uso, lo que hace más difícil dar una cifra mensual fija; Fly.io da más control de infraestructura (regiones, máquinas) a costa de más complejidad operativa, que no se justifica para el volumen de tráfico esperado en un comercio de este tamaño. Ejecutar el backend como funciones serverless en el propio Vercel se descartó por el punto de separación de responsabilidades de 4.2.

4.5 Base de datos y almacenamiento: Supabase

- Un único proveedor resuelve PostgreSQL administrado y almacenamiento de archivos (bucket) con el mismo panel, las mismas credenciales de proyecto y una sola relación comercial, lo que simplifica la operación.
- PostgreSQL es la base natural para un modelo relacional de catálogo (productos, categorías, imágenes, y en la mejora futura pedidos), con soporte completo de claves foráneas, restricciones únicas y transacciones — necesarias para el descuento de stock atómico al confirmarse un pago.
- El plan gratuito (500 MB de base de datos, 1 GB de storage) alcanza con holgura el volumen inicial del proyecto, con un camino de upgrade a plan pago sin migración de proveedor.

Alternativa considerada: una base de datos administrada en Render evitaría un proveedor adicional, pero obligaría a resolver el almacenamiento de imágenes en otro servicio aparte, porque Render no está pensado como CDN de archivos estáticos. Concentrar base de datos y bucket en Supabase es la opción que minimiza la cantidad de proveedores sin sacrificar capacidades.

4.6 Resumen de decisiones

Decisión	Elegido	Fundamento principal
Frontend	Next.js / Vercel	SEO, velocidad móvil, ecosistema para el panel de administración.
Backend	NestJS, servicio independiente	Escalado independiente, estructura modular, mantenibilidad a lo largo de varias entregas.
Hosting backend	Render	Procesos persistentes, precio predecible, mismo flujo de CI/CD que Vercel.
ORM	Drizzle	Control fino de SQL, runtime liviano, ajuste al modelo relacional del proyecto.
Base de datos	PostgreSQL en Supabase	Relacional, transaccional, necesaria para stock y pedidos en la mejora futura.
Almacenamiento	Bucket de Supabase Storage	Mismo proveedor que la base de datos; resuelve las imágenes de producto.

5. Análisis de viabilidad

5.1 Viabilidad técnica

- Las tres tecnologías elegidas (Next.js, NestJS, PostgreSQL/Drizzle) son maduras, con documentación extensa y comunidades grandes; no hay apuesta a tecnología experimental en ningún punto del stack.
- El volumen esperado para un comercio de este tamaño (un catálogo de algunos cientos de productos y un tráfico de decenas de miles de visitas mensuales) está muy por debajo de los límites de cualquiera de los tres proveedores elegidos, incluso en sus planes gratuitos o de entrada.
- El equipo debe manejar dos repositorios y dos despliegues (frontend y backend) en lugar de uno, lo que agrega coordinación pero es un patrón estándar de la industria, con tooling maduro (CI/CD de GitHub, variables de entorno por servicio) para sostenerlo.
- Riesgo técnico principal: CORS y autenticación entre dos orígenes distintos (Vercel y Render). Se mitiga con configuración estándar de CORS en Nest y con tokens JWT en vez de cookies de sesión clásicas, que no dependen de que ambos servicios compartan dominio.

5.2 Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Dos proveedores de hosting (Vercel + Render) en vez de uno	Bajo — más superficie operativa, mismo modelo de CI/CD en ambos	Documentar variables de entorno y URLs de cada servicio en un README de infraestructura del proyecto
El plan gratuito de Render hace spin-down del backend tras inactividad	Medio — la primera consulta del día puede tardar más en responder	Contratar un plan de backend siempre activo antes de publicar en el dominio propio
CORS y autenticación entre dominios distintos	Bajo — mitigable con configuración estándar	CORS explícito en Nest y JWT para las sesiones del panel de administración
El panel de administración maneja credenciales y datos del negocio desde el primer día	Medio — superficie de seguridad mayor que un sitio sin login	Contraseñas con hash (bcrypt), rate limiting en el login, HTTPS obligatorio y validación estricta (Zod / class-validator) en cada endpoint
Volumen de imágenes y consultas crece más rápido de lo esperado	Bajo — los planes gratuitos tienen límites de almacenamiento y de cómputo	Camino de upgrade a planes pagos en Supabase y Render sin migración de proveedor

5.3 Conclusión

El stack definido (Next.js/Vercel para el frontend, NestJS/Drizzle/Render para el backend, PostgreSQL y bucket en Supabase) es técnicamente sólido, usa herramientas maduras y mantiene el volumen esperado del proyecto muy por debajo de los límites de cada proveedor. La separación entre frontend y backend, obligatoria desde esta primera entrega, deja la base preparada para incorporar la mejora futura (pagos, pedidos, roles, estadísticas) sin reescribir lo ya construido.

Anexos

1.1 Repositorio de GitHub

- <https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/tree/main>